A.4. PyTorch

In the previous sections you learned how to use gradient descent to find the minimum of a function, but to do that we needed the gradient. For our simple example, we could compute the gradient with paper and pencil. For deep learning models, that is impractical, so we rely on libraries like PyTorch that provide *automatic differentiation* capabilities that make it much easier.

The basic idea is that in PyTorch we create a *computational graph,* similar to the diagrams we used in the previous section, where relations between inputs, outputs, and connections between different functions are made explicit and kept track of so we can easily apply the chain rule automatically to compute gradients. Fortunately, switching from numpy to PyTorch is simple, and most of the time we can just replace `numpy` with `torch`. Let's translate our neural network from above into PyTorch.

Listing A.3. PyTorch neural network

```
import torch

def nn(x,w1,w2):
    l1 = x @ w1                                    1
    l1 = torch.relu(l1)                            2
    l2 = l1 @ w2
    return l2

w1 = torch.randn(784,200,requires_grad=True)       3
w2 = torch.randn(200,10,requires_grad=True)
```

- ***1*** Matrix multiplication
- ***2*** Nonlinear activation function
- ***3*** Weight (parameter) matrix, with gradients tracked

This looks almost identical to the numpy version except that we use `torch.relu` instead of `np.maximum`, but they are the same function. We also added a `requires_grad=True` parameter to the weight matrix setup. This tells PyTorch that these are trainable parameters that we want to